

INFO0004-2: Project 2

Carré Surfer

Laurent Mathy, Lev Malcev, Gauthier Gain, Elias Piette

April 20, 2026

1 Introduction

The "Carré" is a legendary district in the heart of Liège, famous for its many bars and its ability to attract hundreds of students every evening for a drink (or two, or three, or...) after a long day of lectures. It has been the sacred ground of Liège students since the 1970s, and has survived generations of arguable academic decisions.

An incredible phenomenon can be observed there after midnight: the random walking behavior shown by said students, as they clumsily attempt to make their way home on foot. Scientists have yet to fully explain it, though leading hypotheses involve the Chouffe and a complete misunderstanding of Euclidean geometry.

This phenomenon deserves to be modeled, and that is what this project is about. You are invited to form **groups of two**¹ (so you don't get lost on the way), and to implement a game simulating the "drunken walk" of a student leaving the Carré, encountering various obstacles along the path home...

2 Technical Framework

2.1 Wayland

For this project, you are asked to use **Wayland**². Wayland is a modern protocol for Linux systems; think of it as a middleman between your program and what actually appears on your screen. When your code says "draw a rectangle here", Wayland is the one making sure it ends up in the right place on your monitor.

More specifically, Wayland allows programs (called clients) to communicate with the display server (called the compositor), which is responsible for rendering everything you see on screen and handling input events like keyboard and mouse.

In practice, you will be working with `libwayland-client`, a C library that lets your program connect to the compositor, create a window surface, and draw pixels into it.

2.2 Wrapper Interface

Do not worry if this sounds unfamiliar: you are not expected to become a Wayland expert. We provide you with a simple wrapper that takes care of all the low-level details and offers a

¹If you don't have someone in mind, a forum is open on eCampus; use it to share your availability and email address or Discord tag.

²<https://wayland.freedesktop.org/docs/html/>

student-friendly interface for drawing basic primitives (e.g., pixels, rectangles, lines, circles), handling the event queue, and managing keyboard and mouse inputs.

Internally, it communicates directly with the Wayland compositor for you, and its public API is described in `Window.hpp`. Everything you need to know is in the provided `README.md` file.

In addition to the interface, a `game.cpp` file is provided as a minimal demonstration of the Wayland wrapper: it renders a white square with an orbiting circle that can be moved using the WASD keys or the arrow keys.

2.3 Operating System

We recommend working on an **Ubuntu 25.10 virtual machine** or native installation. Wayland is a Linux native display protocol, it relies on a Linux kernel and a Linux compositor, neither of which are present on macOS or Windows.

Keep in mind that **we will compile and try your program in this environment**, so if you use a different image, make sure that it works in ours as well.

Also, for your own sanity, we recommend that you avoid working directly within the VM. A combination of SSH connections and shared folders is likely to provide the most comfortable working environment. This allows you to use your IDE on your host machine while compiling and running commands on the VM, without having to work within the emulated screen.

3 Gameplay

The protagonist of your game is called Thomas. Thomas is walking back home from the Carré, and is going to run into some bulky items along the way.

Your job is to help him dodge the obstacles that come his way using the arrow keys. In practice, Thomas can move on the screen while the scenery scrolls toward him from the right; the classic endless runner mechanic. Think *Subway Surfers*, but on a dirty Liège sidewalk at 2am.



Figure 1: Example scene from Carré Surfer.

The problem is... Thomas is quite drunk. Despite his efforts, he cannot help but randomly wobble from side to side, and pitifully stumbles on random obstacles.

As an example, in Figure 1, you can see Thomas (30x30 px in the game, though taller in real life), and 4 different types of obstacles³:

1. **Leftover kebab** (in yellow), circles of 15 px diameter,
2. **Garbage** (in gray), of size 30x45 px,
3. **Abandoned Bolt scooter** (in green), of size 15x45 px,
4. **Other drunk students** (in violet), of random size from 20x20 px to 40x40 px.

Note: the lines trailing behind the obstacles in Figure 1 only illustrate the fact that they are moving to the left; they are not to be displayed. Imagine this as a live action screenshot of the game. The white square, Thomas, can move up, down, left, and right.

3.1 Game Mechanics

Obstacles

The obstacles appear every random number of seconds (up to you to calibrate it properly) on the right of the screen, at a random vertical position, and all move at the same constant speed. Each of them has their own probability of appearing:

- Leftover kebab: 20%,
- Garbage: 30%,
- Abandoned Bolt scooter: 10%,
- Other drunk student: 40% (*on Friday night, Rue des Célestines*),

Health Points

Thomas has a number of health points (maximum 100). These points are represented by the game's brightness: if the screen is completely bright (by default), he's completely healthy, and if it's completely black, then it's at 0/100, he passed out.

Every obstacle he runs into has a specific property:

- **For each obstacle** except for kebab leftovers: the screen becomes darker, because he gets hurt.
- Leftover kebab: heals him a little bit (**+10**).
- Garbage: hurts him a little bit (**-10**).
- Abandoned Bolt scooter: hurts him a lot, as he trips and falls to the ground (**-30**).
- Other drunk student: hurts him a bit (**-20**), but he's so embarrassed that **his movements are temporarily reversed** for 1 to 5 seconds (e.g., `ARROW UP` becomes `ARROW DOWN`, `ARROW LEFT` becomes `ARROW RIGHT`, ...).

When Thomas has no health points left, the game is over. The user can then press `SPACE` to restart.

³The sizes are for reference only. The idea is that you can handle them properly in C++, but feel free to adjust them to make the game more enjoyable to play, as long as the dimensions are more or less similar to these ones.

Scoring

The final score corresponds to the distance traveled. At the end of the game, print the number of seconds spent walking to the standard output (e.g., `Thomas walked for 10 seconds!`)

General Behavior

Pressing on arrow keys makes Thomas move for one step. The distance of a step is discrete and yours to define.

Also, since he's drunk, he staggers from side to side every 2 seconds on average (from his point of view, i.e., up or down in the window) for a random number of steps between 1 and 5. Staggering can occur at any point during the game, whether you're moving him or not.

Regarding collisions, whenever Thomas hits anything other than kebab leftovers:

1. **The screen shakes** in a short (less than 1 second) and sharp way, as if the window itself has been bumped.
2. **He becomes invincible** for 2 seconds (because of the adrenaline spike) and blinks quickly during this time.

And if he finds kebab leftovers, he eats them and they disappear.

3.2 Bonus

Feel free to add new features to the game! With Wayland, the sky is the limit; if you make a significant enough contribution, you will be rewarded with bonus points for this project, which accounts for the vast majority of this course's final grade.

4 Guidelines

There are no hard constraints regarding the game's graphical layout. The window size, exact dimensions, and speed settings can be adjusted as you see fit. However, there are a few other directives.

4.1 Game Structure

- You can keep the game at 30 FPS; this is more than enough to compute a frame in C++. The FPS mechanism is already made for you in the example `game.cpp` file.
- At the beginning of the game, give Thomas some space so he can move forward without being immediately showered with obstacles.
- When the game is over, you can simply keep the screen black and print `GAME OVER` with the final score in `stdout`. Then, the player can press `SPACE` to restart, or `ESC` to close the window.

4.2 Readability

Your code **must** be readable by someone who did not write it, including your graders. Examples:

- **Organization:** split your code into logical units. Each class or file should have a clear, single responsibility. A single file called `main.cpp` that contains rendering, input handling, and collision logic all at once is a red flag. If you have to scroll for five minutes to find a function, something is wrong.
- **Naming:** variables, functions, and classes should say what they are. `x` and `tmp` are not names. `playerPosition` and `obstacleSpeed` are. If your name requires a comment to explain it, the name is not good enough.
- **Comments:** well-written code should mostly explain itself. Use comments to explain *why* something is done a certain way, not *what* it does: the code already says what it does. A comment that says `// increment i` above `i++` is either noise, or a Copilot forgotten comment.
- **Consistency:** pick a naming convention (`camelCase`, `snake_case`, etc.), a brace style, and an indentation width, and apply them uniformly throughout your entire codebase.

4.3 Robustness and good practice

The C++ standards and good practices covered throughout the semester are expected to be applied correctly in this project and used when needed. This is an OOP C++ project, not C.

Not exhaustively, this includes using `const` wherever a value is not meant to change, managing memory through smart pointers rather than raw `new` and `delete`, and respecting RAI⁴ so that resources are always properly acquired and released.

It also means choosing the right data structures, writing functions that do one thing and do it well, and never letting undefined behavior slide because "it seems to work". The compiler will not always catch your mistakes, but your graders (probably) will.

4.4 Warnings

Your code must compile **without error or warning** with the commands already given in the reference `Makefile`. However, if there are some obscure Wayland warnings that you cannot get rid of for some reason, and you're sure it's not something dramatic, then you can overlook them.

4.5 Object-oriented principles

This is an object-oriented programming course, and **your project must reflect that**. Your game entities, behaviors, and systems should be modeled using the full range of OOP principles and features available in C++, applied where they make sense.

This guideline is probably **the most important and valuable** of the entire project.

4.6 Gitlab usage

You **must** work together in a [ULiège Gitlab](#) repository that we will ask you to grant us access to. This will allow us to assess the fairness of your individual contributions.

This also implies that the workload should be fairly shared between the two of you, and your commits history will serve as evidence, alongside the oral defense in which both of you will be interviewed (see below).

⁴<https://en.cppreference.com/cpp/language/raii>

4.7 Makefile and Wrapper

You **must** work on top of the provided `Makefile`. It should support separate, incremental compilation (i.e. ; if one file is modified, only that file and the files depending on it should be recompiled). This file is supposed to work and to compile the dummy `game.cpp` without any error or warning on your VM. If it's not the case, make sure you have all of the needed dependencies (including `g++`), which are partially detailed in the `README.md` file.

You *are allowed* to change the content of the provided wrappers if it meets your needs, especially if you would like to add bonus features, but you do not have to.

4.8 Report and AI usage

Please read the explanations below very carefully and approach them with a mature mindset. This is not merely a set of requirements for an academic project, but also a state of mind to be integrated into all your future projects.

The use of AI is **not** prohibited. However, we need a way to check that you have indeed mastered the OOP and C++ concepts learned this semester. That is why we will ask you, on the one hand, to present your work during a short oral presentation (see below), and on the other hand, to **write a short report** (under 5 pages) explaining the overall architecture of your project and the rationale behind your design choices. Notably, we strongly recommend you to provide diagrams to illustrate your classes interaction (e.g., class diagrams).

Why did you build this particular class? What is the reason behind this inheritance? Were global variables really necessary in this specific place? How do you manage the state of all entities over time (position, size, color, etc.)? Are your data structures coherent? How is your memory management going? etc.

Basically, show us that you understand what you are doing, and that your entire project did not arise from a single prompt.

Since the use of AI virtually cannot be prohibited today, and is in fact becoming an almost indispensable tool in the face of competition in the IT industry, this significantly complicates our assessment methods, and we are counting on you to demonstrate that you have mastered the course material as best as you can.

4.9 Evaluation

Your code will be evaluated based on all the above criteria. Failure to comply with any of the points mentioned in **bold face** in this section can result in a mark of zero.

You will present your code and your OOP approach during a 5-minute presentation online demo.

Your final grade will be based on the quality of your code, the way it works, the report, and on the short oral defense.

5 Submission

Projects must be submitted through the submission platform before **May 15th, 23:59 CEST**. Late submissions will be accepted, but will receive a penalty of $2^n - 1$ points (/20), where n is the number of days after the deadline (each day start counting as a full day).

You will submit a `<GROUPID>.tar.xz` archive of a `<GROUPID>` folder containing your C++ source code, and all other files needed for your code to run, and a `Makefile` to compile it (with a simple `make`), where `<GROUPID>` is your group ID on the submission platform.

This archive must also contain a `report.pdf` file, no longer than 3 pages.

As it would be hard to test your game automatically, the submission platform will not do basic checks on your submission. You can submit multiple times.